Annotated review: 4-Qubit QFT in PennyLane

Format: section-by-section review. **Result: derivation and code are correct.** The circuit was built gate by gate in numpy (big-endian, wire 0 = most significant bit) and compared with the 16x16 DFT matrix: identical, and unitary. Remaining points are clarity problems and extra checks.

Section	Verdict	Review note
Eq.1 definition	OK	Standard QFT, big-endian.
Step 1 binary expansions (Eq.2-4)	OK	Fraction notation 0.x_a...x_b defined correctly.
Step 2 product form (Eq.5-9)	OK	x*y/2^n = sum_k y_k x/2^(k+1) is right; the sum over y factorises into n independent 2-term sums.
Step 3 integer-part removal (Eq.10-11)	OK	Integer part is exactly j <= n-2-k; fraction is 0.x_(n-1-k)...x_(n-1).
Eq.12 output state for n=4	OK	Prefactor 1/4 = (1/sqrt 2)^4 correct.
Step 4 H and controlled-phase (Eq.13-15)	OK	phi_(i,j) = 2*pi/2^(j-i+1) = pi/2^(j-i): pi/2, pi/4, pi/8.
Evolution before SWAP (Eq.16)	OK	Wires hold 0.x0x1x2x3, 0.x1x2x3, 0.x2x3, 0.x3 - exactly the reverse of Eq.12.
12-step gate table	OK	4 H + 6 CP + 2 SWAP = 12 gates.
Python code	OK	Loop reproduces the table exactly; SWAPs 0<->3 and 1<->2 are right.

Problems worth fixing in the write-up

1. 'Wire k' means two different things. In Step 3 the bullets 'Wire 0 (k=0): (|0>+e^{2 pi i 0.x3}|1>)' describe the final output (after SWAPs), but in Section 3 'Wire 0' carries 0.x0x1x2x3 before the SWAPs. Same words, opposite states. Rename Step 3 as 'output qubit k' and Section 3 as 'physical wire i (pre-SWAP)'.

2. The ordering rule is never stated. The gates are applied in the order H(i) then CP from j > i, and that order is forced: qubit j must still be in its computational value |x_j> when it acts as a control, so its own Hadamard has to come after all controlled phases that use it as control. Only the code loop shows this, never the text.

3. Eq.14 should say the phase gate is symmetric (control and target interchangeable), so wires=[j,i] or [i,j] gives the same circuit; readers often worry about this.

4. Endianness warning. The derivation is big-endian (wire 0 = MSB), matching PennyLane. Qiskit is little-endian, so the same circuit looks bit-reversed there. One sentence prevents confusion.

5. Unused/odd imports. import json is unused; pennylane.numpy is imported only for np.pi (math.pi works and avoids autograd wrappers).

6. No verification step. A QFT answer should include a test against a reference, see the snippet below.

Extra checks to include (the first compares against PennyLane's own QFT; untested here because PennyLane was not run - the numpy equivalent passed)

```
import numpy as onp
U  = qp.matrix(qft_four_qubits, wire_order=[0, 1, 2, 3])()
Uref = qp.matrix(qp.QFT(wires=[0, 1, 2, 3]))
assert onp.allclose(U, Uref)
N = 16
F = onp.exp(2j*onp.pi*onp.outer(range(N), range(N))/N)/onp.sqrt(N)   # F[y, x]
assert onp.allclose(U, F) and onp.allclose(U.conj().T @ U, onp.eye(N))
```

Possible improvements (and which are worth it)

Idea	Verdict	Detail
Drop the SWAPs, relabel outputs	OK, optional	Saves 2 gates; output is bit-reversed (verified: no-swap circuit differs from the DFT). Fine inside QPE if later gates are relabelled; not fine for a function that must return the true QFT.
Approximate QFT (drop smallest angle pi/8)	WRONG for n=4	Operator-norm error 0.39 (dropping pi/8) and 1.66 (keeping only nearest neighbours). Savings only pay off for large n; do not use here.
General gate count	OK	n H + n(n-1)/2 controlled-phase + floor(n/2) SWAP = 12 for n=4; the circuit is O(n^2) versus O(n 2^n) for a classical FFT of the full amplitude vector.

Idea	Verdict	Detail
Inverse QFT	OK	Reverse gate order and negate angles, or qp.adjoint(qft_four_qubits).
Use qp.QFT(wires=range(4)) directly	OK for production	Not allowed here (task demands elementary gates), but it is the reference for the test above.

Numerical evidence

Check	Result
Doc circuit vs DFT matrix (16x16)	allclose = True; unitary = True
Circuit without SWAPs vs DFT	not equal (bit-reversed output)
Approximate QFT, keep |j-i| <= 3 / 2 / 1	error 0.0 / 0.39 / 1.66